Automated Race Timing

ECMM427 – Group Development Project

The Running Group

May 2025

The Problem: Timing Races Accurately & Affordably

- Many amateur races rely on manual or expensive RFID-based timing systems.
- RFID systems are accurate but costly and require setup overhead.
- Manual timing (clipboards or tablet apps) is error-prone and labour-intensive.
- The goal: an **automated, affordable system** using cameras and computer vision to detect bib numbers and record finish times.

Race Timing Domain

- Used in medium to long distance charity runs, park runs and races.
- Acceptable timing error: 1–2 seconds.
- Key challenge: making a robust, easy-to-use solution for **non-technical event organisers**.

Requirements and Real-World Constraints

- Easy setup, minimal hardware (laptop + cameras).
- Works in poor lighting, rain, cluttered visuals.
- Accurately detects bibs, reads numbers, links runners with times.
- Supports age/gender categories, real-time result uploads.

Development Process

- Agile-inspired workflow with rapid-prototyping and GitHub issues.
- Collaboration tools: GitHub Projects, Trello.
- Early prototypes validated with sample race videos.

System Architecture Overview

- Local pipeline on laptop: video input $\rightarrow$ bib detection $\rightarrow$ OCR $\rightarrow$ timestamping.
- REST API call posts timing data to central Django website.
- Django site hosts leaderboard, user data, event info.

Computer Vision Pipeline

- **Step 1:** Runner and bib detection (YOLO).
- **Step 2:** OCR extraction of bib number (EasyOCR).
- **Step 3:** Timestamp recorded on successful final read.

Live Demonstration

- Runner crosses finish line.
- System detects bib, extracts number, logs timestamp.
- Result appears on website leaderboard.

Deployment and Setup

- Website built in Django, uses SQLite for simplicity.
- Local app runs on organiser's laptop with webcam or action cam feed.
- University hosted: website can be accessed via the University network

Challenges We Tackled

- OCR errors due to occlusions, arms, runner motion.
- Handling non-standard bib placements (e.g., on thigh).
- Variability in font styles and lighting across races.
- System tuning to avoid false positives / misreads.

Future Extensions

- Support for multiple cameras (multi-angle detection).
- Expand to include start timing + split tracking.
- Migration to mobile

What We Learned

- Built real-world vision system under imperfect conditions.
- Gained experience in Django, YOLO, and OCR.
- Balanced engineering decisions with user needs and setup constraints.

Conclusion

- Delivered a low-cost, camera-based race timing system.
- Designed to be used by race organisers, not just tech experts.
- Real-time results, minimal setup, scalable to events of all sizes.